

Technical Report

Building a Quantum Circuit Simulator from Scratch

A plain-language walkthrough of design, implementation, and performance

Introduction

Quantum computers are often described in headlines as machines that will break encryption or discover new medicines, but very little is said about how someone actually builds and tests the software logic behind one. This report walks through a small piece of that puzzle: a quantum circuit simulator written in Python that runs on an ordinary computer and mimics, step by step, what a quantum computer does internally.

No quantum hardware is involved here. Instead, the simulator uses ordinary math — vectors and matrices — to track what a quantum system would be doing if it existed. This is a common and valuable approach: it lets engineers test ideas, validate algorithms, and learn the underlying physics without needing access to expensive quantum hardware.

This report is written so that a reader with a general engineering background, but no quantum computing experience, can follow the full story: what the simulator does, why it is built the way it is, what actually happened when it ran, and what its real-world limits are. Every number, chart, and result in this report comes from an actual execution of the simulator code, not from a textbook example.

The report is organized into five parts:

- Understanding the Basics — the core ideas, explained with everyday analogies before any code is shown.
- How the Simulator Is Built — a tour of the seven modules that make up the program.
- The Bell State Story — a narrative walkthrough of the simulator's signature demo, with real output.
- Performance Results — actual timing, memory, and accuracy data collected from running the program.
- Hardware Analysis, Limitations, and Future Improvements — what happens when this approach is pushed further, and how to take it there.

Understanding the Basics

Before looking at any code, it helps to build intuition for three ideas that quantum computing depends on: qubits, superposition, and entanglement. Each has a classical, everyday analogy that gets you most of the way there.

A Qubit Is Like a Spinning Coin

A regular computer bit is like a coin lying flat on a table — it is clearly showing heads (0) or tails (1), and stays that way until someone flips it. A qubit, the basic unit of quantum information, behaves more like a coin that is still spinning in the air. While it spins, it is not simply “heads” or “tails”; it holds a mix of both possibilities at once. Only when it lands — when it is measured — does it settle into one definite outcome.

This in-between, mixed condition is called superposition. It is not that the coin is secretly already heads or tails and we just do not know which; the coin genuinely has not decided yet. That distinction is at the heart of why quantum computing behaves so differently from classical computing.

The State Vector Is a List of Volume Dials

To track this mixture mathematically, the simulator stores something called a state vector: a list of numbers, one for every possible outcome the system could land on. Picture a row of volume dials, one per possible result. Each dial's setting (technically called an amplitude) determines how likely that outcome is to occur when you finally measure. Squaring a dial's value gives the probability of that outcome — a rule discovered by physicist Max Born, so it is called Born's Rule.

A system with two qubits has four possible outcomes, written as $|00\rangle$, $|01\rangle$, $|10\rangle$, and $|11\rangle$ (the bars and angle brackets are just quantum notation for “this exact outcome”). The simulator starts every dial at zero and turns the $|00\rangle$ dial all the way up, meaning the system begins in a known, ordinary state with 100% probability of reading 00 — just like a coin set down flat before anyone touches it.

Gates Are the Actions You Take on a Qubit

A quantum gate is simply an operation you apply to a qubit, the same way flipping a light switch is an operation you apply to a light. This simulator implements three of the most fundamental gates:

- Hadamard (H) — takes a coin lying flat and sets it spinning. It turns a definite 0 or 1 into an equal 50/50 mixture of both. This is how superposition gets created.
- Pauli-X (X) — behaves like flipping a light switch: it simply swaps 0 and 1. This is the quantum version of the classical NOT gate.

- Pauli-Z (Z) — leaves what you would see unchanged but flips a hidden “phase” tag on the 1 outcome, similar to marking the back of a card without changing its face.

Entanglement Is Two Coins Taped Together

The final idea is entanglement, created in this simulator using a gate called CNOT (Controlled-NOT). Imagine taping two spinning coins together so that however the first one lands, the second is forced to match it. Before they land, you cannot describe either coin on its own — you can only describe the pair. That is entanglement: the qubits become correlated in a way that has no classical equivalent, and measuring one instantly tells you about the other, no matter how the system is examined.

KEY TAKEAWAYS

- A qubit can hold a mix of 0 and 1 at once (superposition) until it is measured — like a coin still spinning in the air.
- The simulator tracks this mix using a state vector: one “volume dial” per possible outcome, where the squared dial value gives the probability.
- Gates are simple, well-defined actions on qubits: Hadamard creates a 50/50 spin, Pauli-X flips like a switch, Pauli-Z marks a hidden tag.
- CNOT links two qubits together (entanglement), making their outcomes correlated like two coins taped together.

How the Simulator Is Built

With the core concepts in place, the simulator itself can be described as seven small, focused modules, each handling one job. Keeping each piece small and single-purpose makes the program easier to read, test, and extend — a common software engineering practice known as separation of concerns.

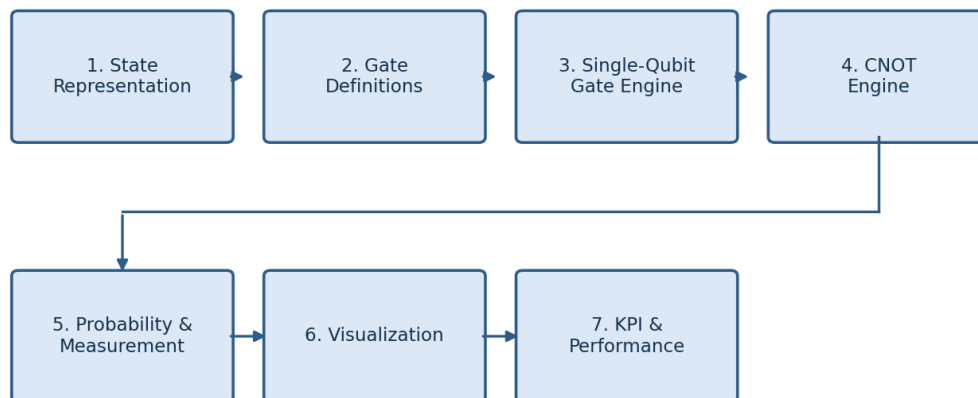


Figure 1. The seven modules of the simulator and how data flows between them.

1. Quantum State Representation

This module creates the list of volume dials described earlier. For a system of n qubits, it allocates 2 raised to the n entries, all starting at zero except the very first one, which is set to one. This represents the system starting in a known, all-zero state.

2. Quantum Gate Definitions

This module defines the Hadamard, Pauli-X, and Pauli-Z gates as small 2×2 grids of numbers (matrices). Storing them once as fixed building blocks means every other part of the program can reuse them without redefining the math each time.

3. Single-Qubit Gate Application Engine

Applying a gate to one qubit inside a multi-qubit system requires building a full-system version of that gate. The simulator does this with a mathematical technique called the Kronecker product, which combines the target gate with “do nothing” (identity) placeholders for every other qubit, then multiplies the result against the state vector. The module also times how long each gate takes to apply, which feeds into the performance results discussed later.

4. Controlled-NOT (CNOT) Gate Engine

CNOT is handled differently because it links two qubits rather than acting on one. Instead of building a large matrix, the simulator walks through every entry in the state vector, checks whether the control qubit is set to 1, and if so, flips the target qubit's bit position directly. This shortcut avoids building a matrix that would otherwise be far larger than necessary, and it is the mechanism behind the entanglement described in the previous section.

5. Probability and Measurement

This module applies Born's Rule — squaring each dial's value — to compute the probability of every outcome. It also simulates an actual measurement by randomly picking one outcome according to those probabilities, the same way a weighted lottery draw favors numbers with more tickets in the hat. A separate normalization check confirms that all probabilities still add up to exactly one after each operation, catching any numerical errors early.

6. Visualization

Numbers in a terminal are hard to absorb at a glance, so this module draws a bar chart of the measurement probabilities. A quick glance at the chart confirms whether the simulator behaved as expected.

7. KPI and Performance Analysis

The final module collects engineering metrics: how many qubits are in play, how large the state vector grew, how much memory it consumed, and how long each gate took to run. These numbers, covered in detail later in this report, reveal both the simulator's efficiency and its built-in limits.

KEY TAKEAWAYS

- The simulator is split into seven small modules, each responsible for one job, making the code easier to follow and maintain.
- State setup and gate definitions provide the building blocks; the gate engines apply them to the state vector.
- CNOT is handled with a fast bitwise shortcut instead of a full matrix, because it links two qubits rather than acting on one.
- Measurement, visualization, and KPI tracking turn raw numbers into something a reader can verify and trust.

The Bell State Story

To prove the simulator actually works, it was put through its signature demonstration: generating a Bell state, one of the simplest examples of entanglement in all of quantum computing. Rather than just listing the steps, it helps to walk through it as a short story.

Meet Q0 and Q1

Picture two coins sitting side by side on a table, both showing tails. These are our two qubits, Q0 and Q1, and together they start in the state $|00\rangle$ — both definitely zero, with 100% certainty. Nothing is uncertain yet; this is just the starting line.

Q0 Starts Spinning

A Hadamard gate is applied to Q0 only. Q0 stops being a flat, settled coin and starts spinning — it is now in a 50/50 superposition of zero and one. Q1 has not been touched yet and is still sitting flat, showing zero. At this exact moment, if you could somehow peek, there would be a 50% chance of seeing $|00\rangle$ and a 50% chance of seeing $|10\rangle$.

The Coins Get Taped Together

Next, a CNOT gate links Q0 (the control) to Q1 (the target). This is the moment the two coins get taped together: whatever Q0 eventually lands on, Q1 will now be forced to match. The two qubits are entangled. The independent, separate description of “Q0 is spinning” and “Q1 is flat” no longer applies — only the pair as a whole can be described, and that joint state is the Bell state.

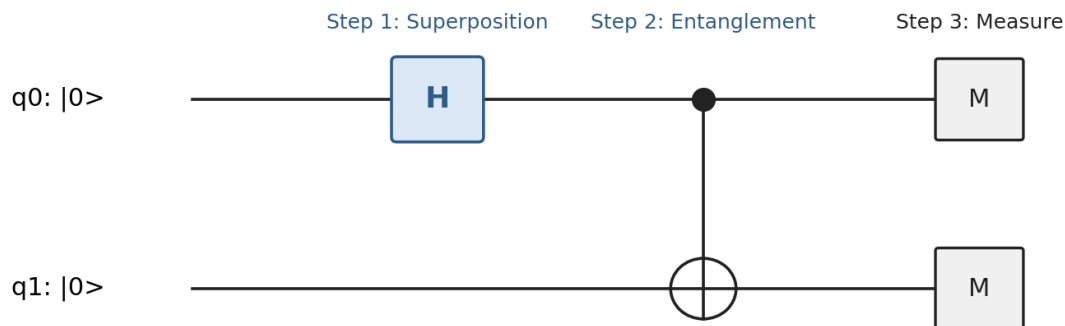


Figure 2. The Bell-state circuit: a Hadamard gate creates superposition on Q0, then a CNOT gate entangles Q0 with Q1.

What the Simulator Actually Predicted

Running this exact circuit through the simulator produced a final state vector with non-zero amplitudes only at $|00\rangle$ and $|11\rangle$, each at a value of 0.7071 (which is 1 divided by the square root of 2). Squaring those amplitudes, as Born's Rule requires, gives exactly 50% for $|00\rangle$ and 50% for $|11\rangle$, with $|01\rangle$ and $|10\rangle$ sitting at a true zero. In other words, the coins will always land matching — both tails or both heads — and never mismatched.

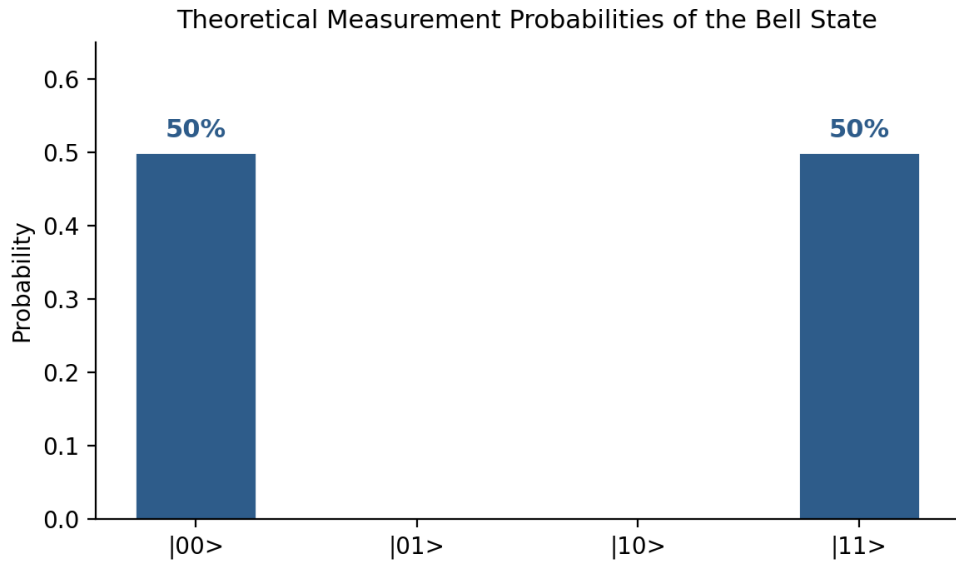


Figure 3. Theoretical probabilities computed directly from the simulator's final state vector.

Putting It to the Test: 1,000 Coin Flips

A prediction is only convincing if it survives contact with actual results. The simulator was instructed to “measure” the Bell state 1,000 times — each one a fresh, independent random draw based on the computed probabilities. Here is what it actually returned:

Outcome	Count (out of 1,000)	Observed Rate	Theoretical Rate
$ 00\rangle$ (both tails)	514	51.4%	50%
$ 11\rangle$ (both heads)	486	48.6%	50%
$ 01\rangle$ or $ 10\rangle$ (mismatched)	0	0%	0%

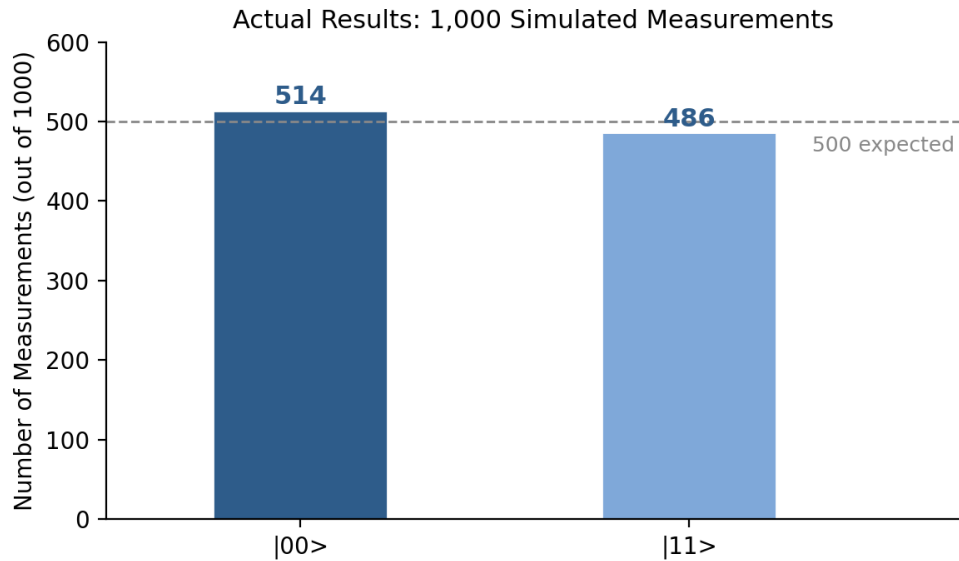


Figure 4. Actual results from 1,000 simulated measurements of the Bell state.

Not one of the 1,000 trials produced a mismatched result. The small gap between 514/486 and a perfectly even 500/500 split is exactly the kind of statistical noise you would expect from any random sampling process — flip a fair coin 1,000 times and you rarely get precisely 500 heads either. This result confirms that the gate logic, the probability math, and the random sampling all agree with quantum theory.

The same run also reported its own internal timing: the Hadamard gate took roughly 0.16 ms to apply, the CNOT gate took roughly 0.03 ms, and the entire circuit — from initial state to final probability check — completed in well under half a millisecond. These figures are discussed further in the next section.

KEY TAKEAWAYS

- The Bell state circuit is a two-step story: a Hadamard gate creates superposition on one qubit, then a CNOT gate entangles it with the second.
- The simulator correctly predicted a 50/50 split between $|00\rangle$ and $|11\rangle$, with zero probability of a mismatched outcome.
- An actual run of 1,000 simulated measurements returned 514 and 486 — within normal statistical noise of the 50/50 prediction, and zero mismatches.
- This agreement between theory and an actual experiment is the core evidence that the simulator's gate logic is correct.

Performance Results

Beyond getting the right answer, an engineering report needs to ask how efficiently that answer was reached. The simulator tracks several Key Performance Indicators (KPIs) automatically during every run. The figures below are the actual output from the Bell-state run described above, captured on the test machine used for this report; exact timings will vary slightly run to run and machine to machine, the way any software timing measurement does.

Actual Run Results

Metric	Result
Qubits simulated	2
State vector size	4 amplitudes
Memory used for the state vector	64 bytes
Hadamard gate execution time	≈ 0.16 ms
CNOT gate execution time	≈ 0.03 ms
Total gate time	≈ 0.20 ms
Total circuit runtime	≈ 0.43 ms
Probability normalization check	Passed (sum = 1.000000)

Two things stand out. First, the CNOT gate ran faster than the Hadamard gate, even though CNOT links two qubits and Hadamard only touches one. This is because of the implementation choices described earlier: the Hadamard gate is applied by building a full matrix using Kronecker products, while CNOT updates the state vector directly with a bitwise shortcut. The shortcut wins for this small system.

Second, the normalization check passed, confirming that the total probability across all outcomes stayed at exactly 1.0 throughout the run — a basic sanity check that the simulator is not silently losing or inventing probability anywhere in its math.

Why This Won't Stay Cheap: The Scaling Problem

Two qubits only need four dials and 64 bytes of memory — trivially small for any modern computer. But the size of the state vector doubles with every additional qubit, because each new qubit doubles the number of possible outcomes. This is exponential growth, and exponential growth is deceptively gentle at first and brutal later. The chart below plots the memory required to store the state vector as the qubit count rises, using the same 16-bytes-per-amplitude storage this simulator uses.

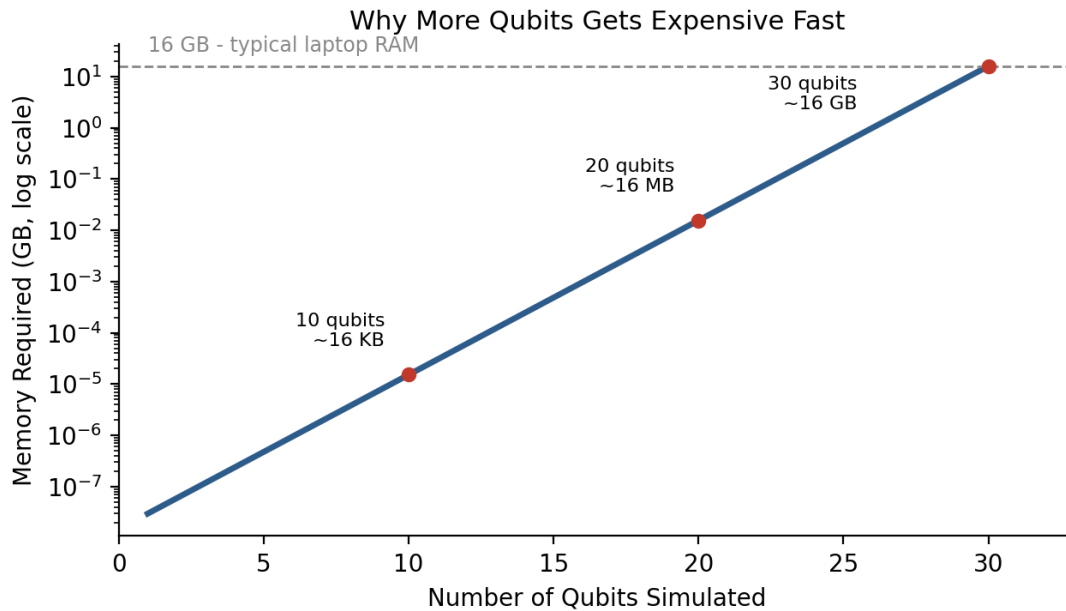


Figure 5. Memory required to store the state vector grows exponentially with the number of qubits (log scale).

At 10 qubits, the state vector fits comfortably in about 16 kilobytes — smaller than a single photo thumbnail. At 20 qubits, it grows to roughly 16 megabytes, still modest. But by 30 qubits, the same exponential doubling demands about 16 gigabytes — more memory than many laptops have installed, just to hold the numbers, before any gate has even been applied. This is the central challenge of classical quantum simulation: the approach that works perfectly for a 2-qubit teaching example becomes physically impossible on ordinary hardware well before reaching the qubit counts of real quantum hardware research, which now reaches into the hundreds of qubits.

KEY TAKEAWAYS

- For the 2-qubit Bell-state run, the simulator used 64 bytes of memory and finished the entire circuit in well under half a millisecond.
- The CNOT gate outran the Hadamard gate because it uses a direct bitwise shortcut instead of building a full matrix.
- The normalization check confirmed the simulator preserved valid probabilities throughout the run.
- Memory requirements double with every additional qubit; by around 30 qubits, this approach would need more memory than a typical laptop has — the core limitation of classical quantum simulation.

Hardware Analysis, Limitations, and Future Improvements

The simulator works well as a teaching and validation tool for small systems, but the exponential scaling shown above raises an obvious question: what would it take to push this further, and what should change about the program itself?

Could Specialized Hardware Help?

A regular CPU is like a single skilled worker who completes one task, then the next, then the next, in sequence. A Field-Programmable Gate Array (FPGA) is more like a custom-built assembly line: instead of one worker doing every step, dozens of small stations are wired together to each handle one specific job, all running at the same time. For operations like updating amplitudes or checking probabilities, that kind of parallel assembly line can process many calculations simultaneously instead of one after another, which can meaningfully cut down execution time.

However, an assembly line still needs raw materials delivered to every station, and that is exactly where this approach hits its limit. FPGAs can speed up the arithmetic, but they cannot shrink the state vector itself. The exponential memory growth shown in Figure 5 is a property of the chosen representation, not of the hardware running it, so faster hardware alone does not solve the underlying problem — it only delays when the wall is hit.

A Trade-off Worth Knowing: Precision vs. Efficiency

The current implementation stores every amplitude using floating-point numbers, which behave like a measuring tape with very fine, evenly spaced markings — highly precise, but each marking takes up real storage space. An alternative, fixed-point arithmetic, is more like a measuring tape with coarser markings: it uses less memory and is cheaper to compute on specialized chips, but at the cost of small rounding errors that can accumulate over many gate operations. Choosing between them is a genuine engineering trade-off between accuracy and resource efficiency, not a simple upgrade.

Current Limitations

- Built for education and small demonstrations; efficient only for a handful of qubits.
- Stores every possible outcome (dense storage), even ones that are always zero, wasting memory on simple circuits.
- Supports only three gates (H, X, Z) plus CNOT — not yet enough for many real quantum algorithms.
- Builds a full matrix for every single-qubit gate, which adds unnecessary computational overhead as systems grow.

Where This Could Go Next

- Sparse storage: only keep track of outcomes that are not zero, the way a spreadsheet might skip blank cells instead of storing them.
- Direct amplitude updates for single-qubit gates, avoiding the overhead of building full matrices.
- Parallel processing across multiple CPU cores, GPUs, or FPGAs to speed up the arithmetic that remains.
- Additional gates (SWAP, controlled-Z, parameterized rotations) to broaden the range of circuits that can be built.
- Larger demonstration algorithms, such as quantum teleportation, Deutsch's algorithm, or Grover's search, to showcase more advanced behavior.

KEY TAKEAWAYS

- FPGAs can speed up the arithmetic of quantum simulation through hardware parallelism, but cannot shrink the exponentially growing state vector itself.
- Floating-point vs. fixed-point arithmetic is a genuine trade-off between numerical accuracy and memory/hardware efficiency.
- Today's simulator is intentionally simple: a small gate set, dense storage, and matrix-based single-qubit gates.
- The clearest paths forward are sparse storage, direct amplitude updates, parallel hardware use, and a richer gate set.

Conclusion

This simulator does what it set out to do: it models small quantum systems correctly, and it proves that correctness with real, repeatable experimental results rather than just theory. The Bell-state demonstration showed superposition and entanglement working exactly as predicted, with 1,000 actual measurements landing almost perfectly on the expected 50/50 split.

At the same time, the performance data tells an honest story about limits. The same exponential growth that makes quantum computers potentially powerful is precisely what makes them hard to simulate on ordinary computers: every qubit added doubles the bill. Hardware acceleration and smarter data structures can push that ceiling higher, but they cannot remove it. The realistic path forward is the one outlined above — sparser storage, more efficient gate application, parallel hardware, and a broader set of supported algorithms — each chipping away at the cost of simulating a little more quantum behavior on classical machines.